COMPUTER ENGINEERING

QUEZON CITY

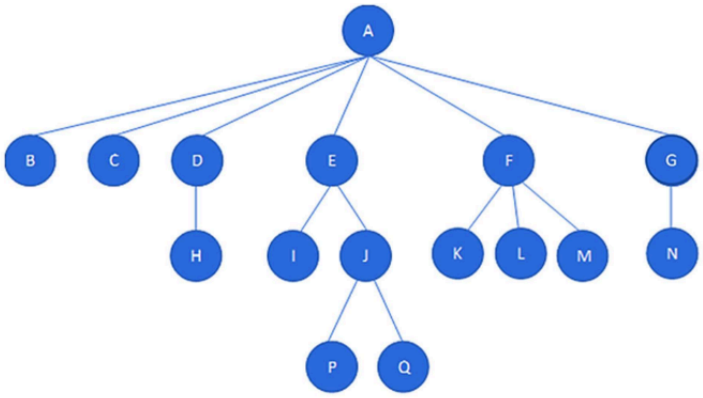
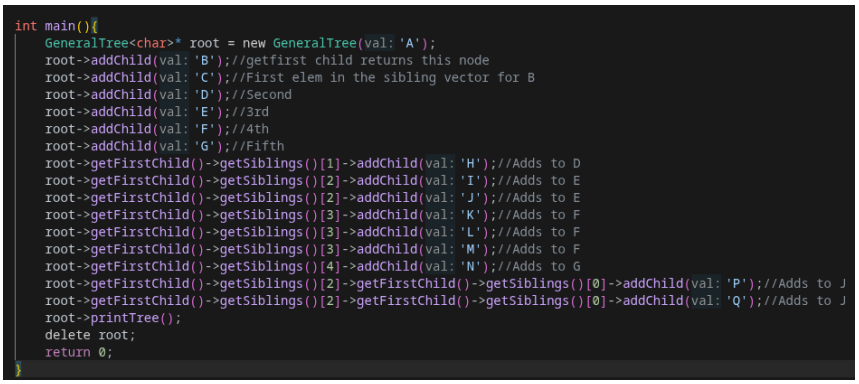
COURSE CODE/TITLE:	CPE 010 - Data Structures and Algorithms	**SECTION:**	CPE21S2
DATE PERFORMED:	9/18/23	**DATE SUBMITTED:**	9/23/26
NAME:	Condino, Niel Vincent B.	**INSTRUCTOR:**	Engr. Jimlord M. Quejado

ACTIVITY NO. 9.1 Tree ADT

1. PROCEDURE OUTPUT

ILO A: Create C++ code to implement a general tree, binary tree, and binary search tree

Table 9.1:General Tree Creation

Tree Representation	
Code	

```
int main(){
    GeneralTree<char>* root = new GeneralTree(val: 'A');
    root->addChild(val: 'B');//getfirst child returns this node
    root->addChild(val: 'C');//First elem in the sibling vector for B
    root->addChild(val: 'D');//Second
    root->addChild(val: 'E');//3rd
    root->addChild(val: 'F');//4th
    root->addChild(val: 'G');//Fifth
    root->getFirstChild()->getSiblings()[1]->addChild(val: 'H');//Adds to D
    root->getFirstChild()->getSiblings()[2]->addChild(val: 'I');//Adds to E
    root->getFirstChild()->getSiblings()[2]->addChild(val: 'J');//Adds to E
    root->getFirstChild()->getSiblings()[3]->addChild(val: 'K');//Adds to F
    root->getFirstChild()->getSiblings()[3]->addChild(val: 'L');//Adds to F
    root->getFirstChild()->getSiblings()[3]->addChild(val: 'M');//Adds to F
    root->getFirstChild()->getSiblings()[4]->addChild(val: 'N');//Adds to G
    root->getFirstChild()->getSiblings()[2]->getFirstChild()->getSiblings()[0]->addChild(val: 'P');//Adds to J
    root->getFirstChild()->getSiblings()[2]->getFirstChild()->getSiblings()[0]->addChild(val: 'Q');//Adds to J
    root->printTree();
    delete root;
    return 0;
}
```

The code above creates the tree illustrated in the table. The tree uses the class GeneralTree to implement this data structure. The addChild() method adds a child to the node where it was called. It first checks if the node has a child. If it does not, a node is created and linked using a pointer, then stored in the child attribute of the parent node; If it does, then it creates a new node and stores it in a vector of pointers for the siblings of the first child. The getSiblings() function returns a dynamic array where the siblings of the first child are stored. These functions run at O(1) complexity.

Table 9-2:Node and Depth of each node

Node	Height	Depth
A	3	0
B	2	1
C	2	1
D	2	1
E	2	1
F	2	1
G	2	1
H	1	2
I	1	2
J	1	2
K	1	2
L	1	2
M	1	2
N	1	2
P	0	3
Q	0	3

This table represents each node's place in the tree. Using the "depth" sorting, it starts with the root "A", where the depth of that node is 0 and each level below that is depth + 1. The height, however, starts at 0 at the lowest depth node. From this lowest depth, each level above it increments the height by one. The table above shows each node's respective height and depth.

ILO B: Create C++ code for implementation of tree traversal methods such as pre-order, in-order, and post-order traversal

Table 9-3:Tree Traversal

Pre-order	A, B, C, D, H, E, I, J, P, Q, F, K, L, M, G, N
Post-order	B, C, H, D, I, P, Q, J, E, K, L, M, F, N, G, A
In-order	B, A, C, H, D, I, E, P, J, Q, K, F, L, M, N, G

The values in the table above show the expected output of each traversal method. Each of the outputs in the tables is manually enumerated using the algorithms for each method. This table will be used later to check the output of the next table to make sure that the code works as expected.

Table 9-4:Traversal code implementation

Code

```
template<typename T>
template<typename F>
void GeneralTree<T>::preOrder(F funcCall) const{
    funcCall(this);
    if (!isLeaf()){
        //visit child
        child->preOrder(funcCall);
        //then child siblings
        for (const auto* sibling:child->siblings){
            sibling->preOrder(funcCall);
        }
    }
}

template<typename T>
template<typename F>
void GeneralTree<T>::inOrder(F funcCall) const{
    if (!isLeaf()){
        child->inOrder(funcCall);
    }
    funcCall(this);
    if (!isLeaf()){
        for (const auto* sibling:child->siblings){
            sibling->inOrder(funcCall);
        }
    }
}

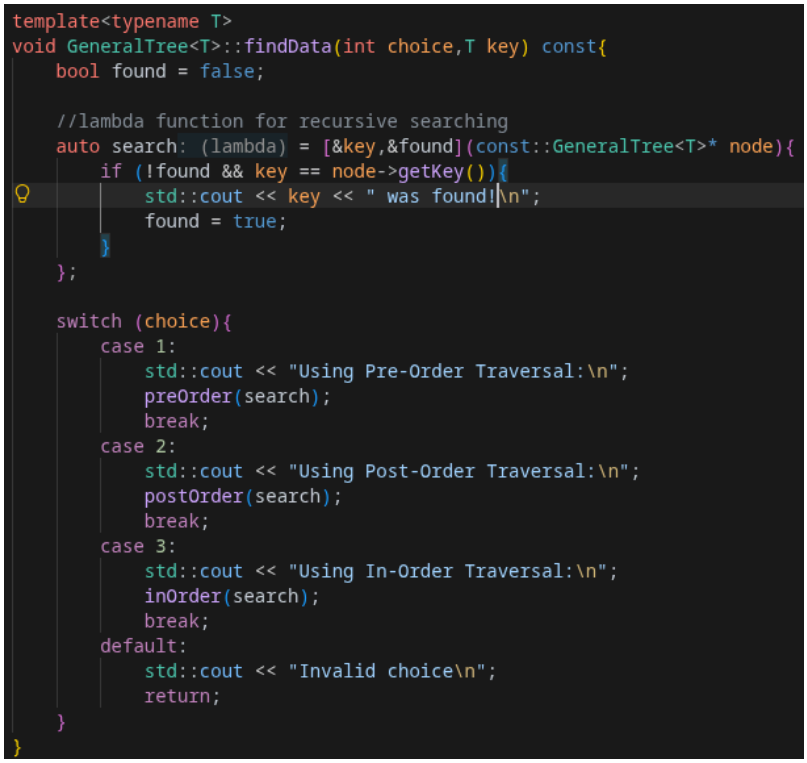
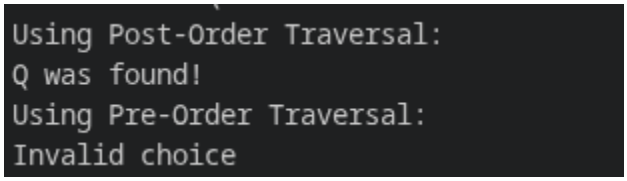
template<typename T>
template<typename F>
void GeneralTree<T>::postOrder(F funcCall) const{
    if (!isLeaf()){
        child->postOrder(funcCall);
        for (const auto* sibling:child->siblings){
            sibling->postOrder(funcCall);
        }
    }
    funcCall(this);
}
```

Output

```
Pre-Order:
A B C D H E I J P Q F K L M G N
In-Order:
B A C H D I E P J Q K F L M N G
PostOrder:
B C H D I P Q J E K L M F N G A
```

Cross-referencing the code output with the manually evaluated output from the last table, they both match in their values; this signifies the code implementation was able to meet the required output. Like a binary search tree, it has left and right pointers. The left points to the first child, while the “right” is a vector that holds a vector of pointers for that child's siblings. Their traversal algorithm follows the same algorithm as that of a binary search tree, with the counterpart traversing the first child if left is chosen and looping through the vector of siblings when the algorithm says to traverse the right. This runs at O(n) runtime, where n is the number of nodes in the tree.

Table 9-5:findData function implementation

Function Code	
Driver Code	
Output	

```template<typename T>
void GeneralTree<T>::findData(int choice,T key) const{
    bool found = false;

    //lambda function for recursive searching
    auto search: (lambda) = [&key,&found](const::GeneralTree<T>* node){
        if (!found && key == node->getKey()){
            std::cout << key << " was found!\n";
            found = true;
        }
    };

    switch (choice){
        case 1:
            std::cout << "Using Pre-Order Traversal:\n";
            preOrder(search);
            break;
        case 2:
            std::cout << "Using Post-Order Traversal:\n";
            postOrder(search);
            break;
        case 3:
            std::cout << "Using In-Order Traversal:\n";
            inOrder(search);
            break;
        default:
            std::cout << "Invalid choice\n";
            return;
    }
}
```

```//find data test
std::cout << '\n';
root->findData(choice: 2, key: 'Q');
root->findData(choice: 1, key: 'Z');
root->findData(choice: 4, key: 'D');
```

```Using Post-Order Traversal:
Q was found!
Using Pre-Order Traversal:
Invalid choice
```

The code implementation has successfully reached the desired output of the task. The function takes in a choice argument and the desired key. To choose the right algorithm, a switch case is utilized. A lambda function is used to search for the right element given as the argument of the function. It uses a bool flag “found” and the key in its capture clause. This checks if the element is not found and the key matches the node’s current key. This algorithm runs in O(n) time because it runs like a linear search.

Table 9-6: New child findData() testing

Code	`//add O as a child of G`
	`root->getFirstChild()->getSiblings()[4]->addChild(val: 'O');`
	`root->findData(choice: 3, key: 'O');`

Output	`Using In-Order Traversal:` `O was found!`

As expected, after adding “O” as the child of “G”, the function was still able to find the node O in the tree. This means the function still works even if the tree’s nodes change mid-execution.

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

```cpp
#include <iostream>
#include "BST.h"

int main(){
    //create ugat
    BinarySearchTree<int> tree(val: 2);
    const int size = 7;
    int treeArr[size] = {[0]=3,[1]=9,[2]=18,[3]=0,[4]=1,[5]=4,[6]=5};

    for (int i = 0; i < size; i++){
        //std::cout << treeArr[i] << '\n';
        tree.insert(val: treeArr[i]);
    }

    std::cout << "preOrder:";
    tree.preOrder();
    std::cout << '\n';
    std::cout << "inOrder:";
    tree.inOrder();
    std::cout << '\n';
    std::cout << "postOrder:";
    tree.postOrder();
    return 0;
}
```

Figure 1:Tree driver code

Above is the binary search tree implementation from the given values. It constructs a tree by looping through the array, then inserting it into the tree. This runs in $O(n^2)$ as the loop runs O(n) times and the insert method of the tree runs in O(n) time. The tree is then checked for the values using the different traversal algorithms, which will be discussed in the next figures

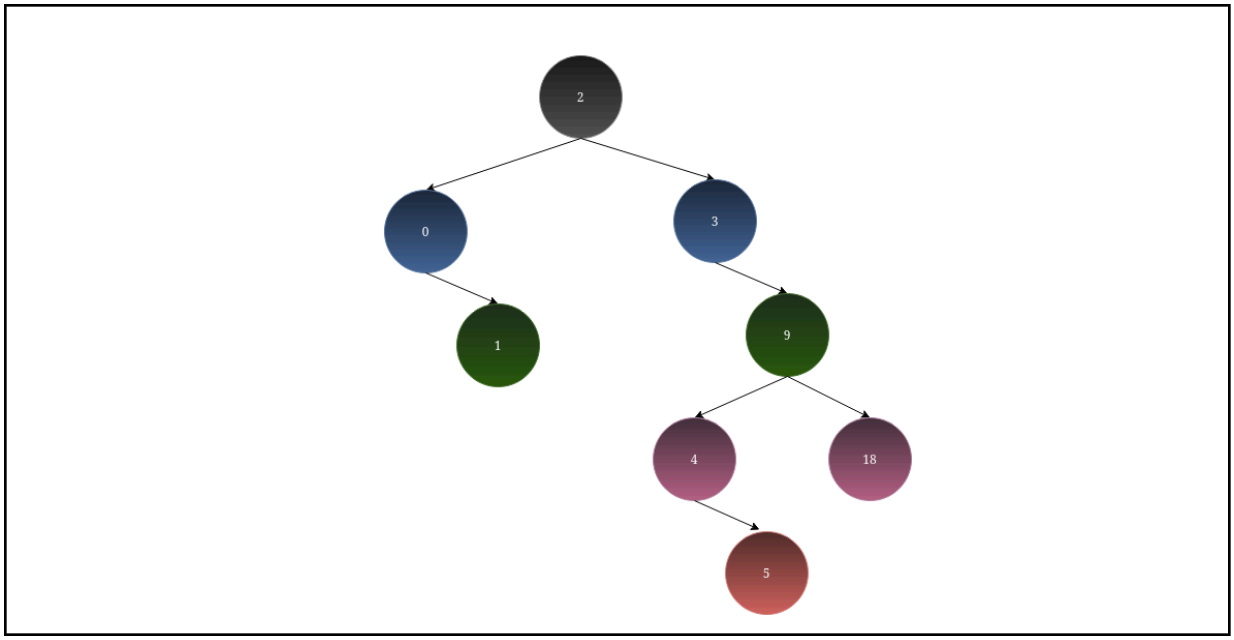


Figure 2:Tree Diagram

This is a drawing representation of the created tree. This better visualizes how the nodes are connected in a tree-like structure in memory. Each root node has at most 2 children: the left child's value is less than the root, and the right child's value is greater than the root.

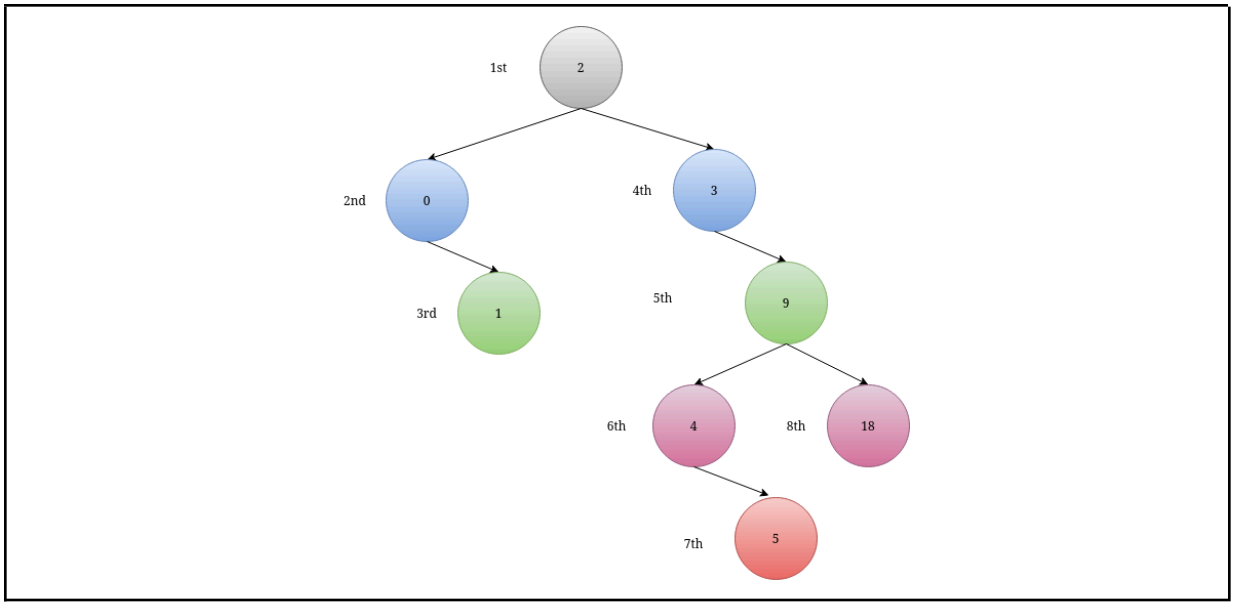


Figure 3: Pre-order traversal

This figure shows the order of traversal using the preorder traversal algorithm. This prints the root node first, then checks the left node, if it exists, then the right node, if it also exists. It can be summarized as "Root, Left, Right".Using this order, the following output is expected: 2->0->1->3->9->4->5->18.

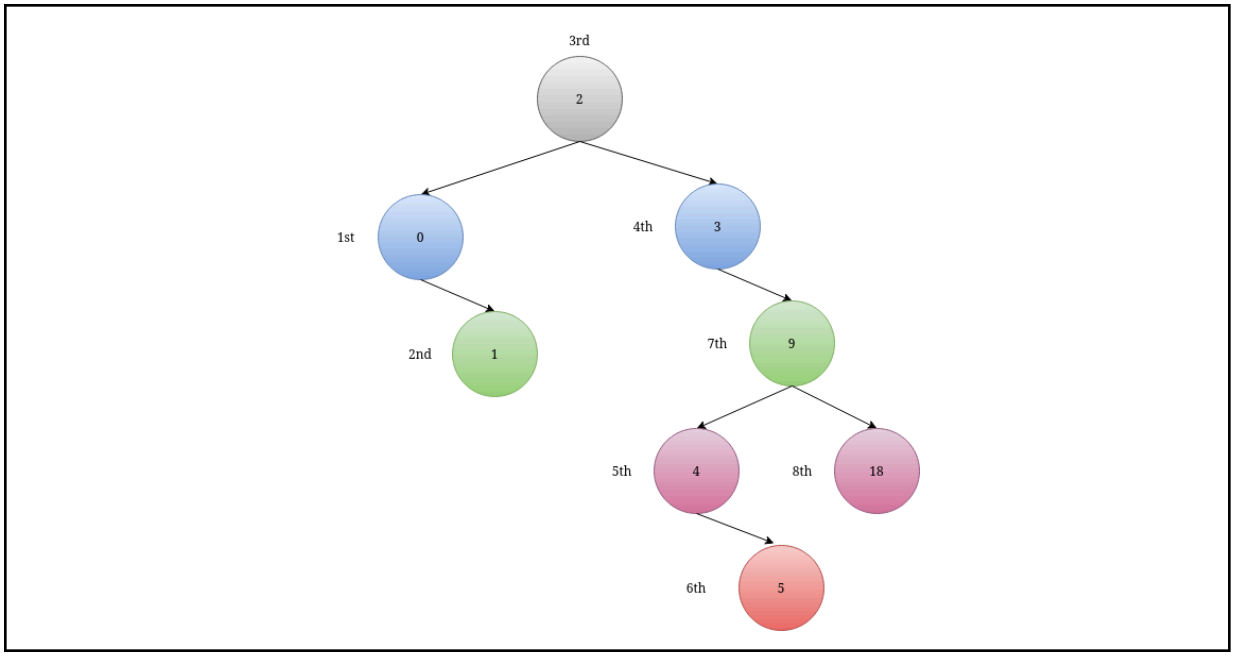


Figure 4: In-order traversal

The figure above shows the order of traversal using the in-order traversal algorithm. It first checks the leftmost node, the root of the subtree, then the rightmost node. It can be summarized as “Left, Root, Right”. Following this order, the output is: 0->1->2->3->4->5->9->18. Looking at the expected output, it appears that the values are sorted in ascending order

.

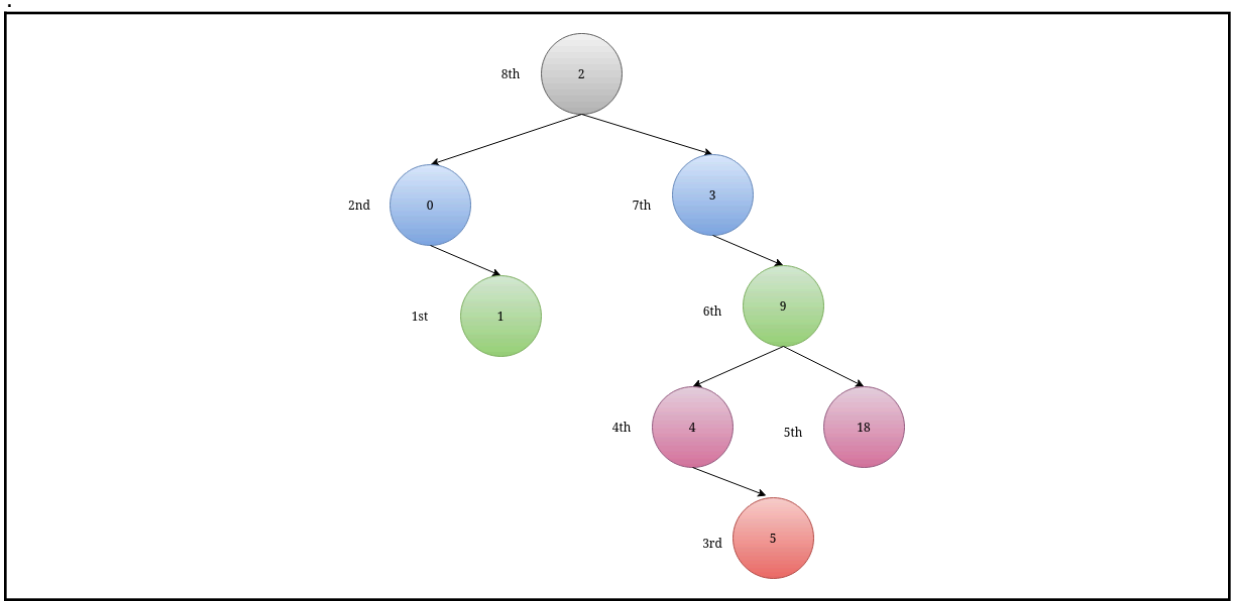


Figure 5: Post-order traversal

The figure above shows the order of traversal using the post-order traversal. It first checks the leftmost node, then the rightmost, and lastly the root. It can be summarized as “Left, Right, Root”. Following this order, the output is expected to look like this: 1->0->3->4->18->->9->3->2.

```
//L,R,RI
template<typename T>
void BinarySearchTree<T>::inOrder(){
    if (left != nullptr){
        left->inOrder();
    }
    std::cout << key << ' ';
    if (right != nullptr){
        right->inOrder();
    }
}
```

```
inOrder:0 1 2 3 4 5 9 18
```

Figure 6: In-order traversal code

The code aggressively traverses the leftmost node first, prints the key, then aggressively goes to the rightmost node. This follows the left, root, right approach. This way, the tree is traversed in an ascending manner. When it traverses each node, the function is called recursively. This runs in O(n) time complexity. The output is the same order as illustrated in Figure 4.

```
//R L RI
template<typename T>
void BinarySearchTree<T>::preOrder(){
    std::cout << key << ' ';
    if (left != nullptr){
        left->preOrder();
    }
    if (right != nullptr){
        right->preOrder();
    }
}
```

```
preOrder:2 0 1 3 9 4 5 18
```

Figure 7:Pre-Order traversal code

The code prints the key, aggressively traverses the leftmost node first, then aggressively goes to the rightmost node. This follows the root, left, right approach. When it traverses each node, the function does this recursively. This runs at O(n) time complexity. The output visualization is illustrated in the previous figure, Figure 3.

```
//L,RI,R
template<typename T>
void BinarySearchTree<T>::postOrder(){
    if (left != nullptr){
        left->postOrder();
    }
    if (right != nullptr){
        right->postOrder();
    }
    std::cout << key << ' ';
}
```

```
postOrder:1 0 5 4 18 9 3 2
```

Figure 8: Post-Order traversal

The code traverses to the leftmost node first, then goes to the rightmost node, and then prints the key. This follows the left, right, root approach. To traverse each node, the function is invoked recursively. This runs at O(n) time complexity. The visualization of this traversal can be seen back in the previous figure, Figure 5.

Conclusion:

In this activity, the concept of trees is discussed. This data structure is called this because it functions like a tree in the physical world; it consists of a root, branches, and leaves. Unlike past structures like arrays, linked lists, queues, and stacks, where it stores data in a line, trees store values in different levels that can be classified as depth or height. I was able to construct a general tree from the procedure part of this activity. To implement this, I looked online, tried to understand it, then constructed the class on my own. I am satisfied with how I implemented the traversal method to take a function as its argument to execute different work for each traversal type. In the supplementary, I was able to understand and implement a binary search tree. This is just a subcategory of the tree where a node can only have a max of 2 children; The left child is always less than the root, and the right node being greater than the root node. I was able to implement methods such as inserting a value into it. This is also where the traversal algorithm such as inorder , preorder, and post orderis explained, and I feel I have a grasp on it. I look forward to learning about graphs as I found these data structures interesting.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **Grammarly**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: **Grammar Correction**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of

concepts, etc.

Extent of Use: **Basic subscription, highlights wrong grammar**

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.